

# Multi-LLM Agent System for Automatic Information Search and Synthesis

---

Όνομα: Αθανάσιος

Επώνυμο: Εμμανουηλίδης

AEM: 211

## Περιεχόμενα

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Multi-LLM Agent System for Automatic Information Search and Synthesis..... | 1  |
| 1. Εισαγωγή.....                                                           | 2  |
| 2. Επισκόπηση Αρχιτεκτονικής Συστήματος.....                               | 4  |
| 2.1 High-level pipeline.....                                               | 4  |
| 2.2 Αντιστοίχιση με κλασικές έννοιες πολλαπλών πρακτόρων.....              | 4  |
| 2.3 Σχεδιασμός κοινόχρηστης κατάστασης.....                                | 4  |
| 3. Οι Πράκτορες.....                                                       | 5  |
| 3.1 Web Agent.....                                                         | 5  |
| 3.2 RAG Agent.....                                                         | 5  |
| 3.3 Voting Sub-graph.....                                                  | 6  |
| 3.4 Synthesizer Agent.....                                                 | 7  |
| 4. Βασικές Αποφάσεις Υλοποίησης.....                                       | 7  |
| 4.1 Απενεργοποίηση παράλληλων κλήσεων εργαλείων.....                       | 7  |
| 4.2 <code>return_direct=True</code> είναι απαραίτητο αλλά όχι επαρκές..... | 8  |
| 4.3 <code>InjectedState</code> ως κανάλι δεδομένων.....                    | 8  |
| 4.4 <code>TypedDict</code> αντί <code>Pydantic</code> .....                | 8  |
| 4.5 Ροή του γραφήματος για σχόλια στο περιβάλλον χρήστη.....               | 8  |
| 4.6 Εξαγωγή τελικής απάντησης.....                                         | 9  |
| 5. Evaluation: Multi-agent vs. Single-agent Baseline.....                  | 9  |
| 6. Η Στρατηγική Ψηφοφορίας: Παράμετροι και Αιτιολόγηση.....                | 10 |
| 7. Streamlit User Interface.....                                           | 10 |
| 8. Συμπεράσματα.....                                                       | 11 |
| Appendix A - Project Structure.....                                        | 12 |
| Appendix B - Sample Run Log (Excerpt).....                                 | 12 |

## 1. Εισαγωγή

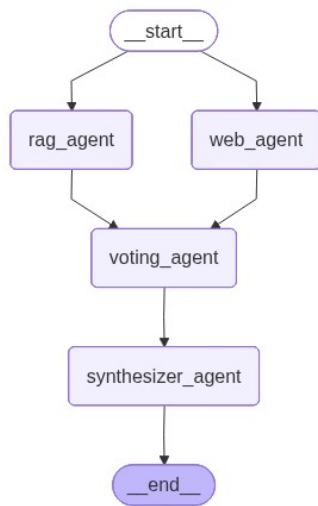
Στόχος αυτού του project είναι ο σχεδιασμός και η υλοποίηση ενός συστήματος πολλαπλών πρακτόρων σε Python, όπου θα υπάρχουν πράκτορες με καθορισμένους ρόλους με σκοπό να συνεργάζονται για να απαντήσουν σε σύνθετα ερωτήματα. Για παράδειγμα, τεχνικά ερωτήματα σχετικά με την επιστημονική βιβλιογραφία. Το σύστημα ακολουθεί τη ροή εργασίας που προτείνεται από την εργασία: Ο ερευνητής συλλέγει → Ο κριτικός αξιολογεί → Ο συνθέτης συνθέτει → Ο ελεγκτής γεγονότων επαληθεύει, με έναν μηχανισμό επίλυσης διαφωνιών μεταξύ των πρακτόρων. Το συγκεκριμένο σύστημα υλοποιήθηκε χρησιμοποιώντας LangGraph για ενορχήστρωση βασισμένη σε γραφήματα και LangChain για το LLM και τα εργαλεία, με το OpenAI API ως τον πάροχο για το μοντέλο. Το UI που βλέπει ο χρήστης είναι κατασκευασμένο με Streamlit. Αυτή η έκθεση καταγράφει την αρχιτεκτονική, τις αποφάσεις σχεδιασμού που λαμβάνονται σε κάθε βήμα, τη στρατηγική επίλυσης διαφωνιών (ένα πρωτόκολλο ψηφοφορίας με τρεις πράκτορες κριτών και έναν βαθμονομητή), το mapping μεταξύ των framework structures και των κλασικών εννοιών πολλαπλών πρακτόρων του μαθήματος (BDI, μετάδοση μηνυμάτων τύπου FIPA, Contract Net), τις δυσκολίες που αντιμετωπίστηκαν και τα τελικά αποτελέσματα σε σύγκριση με μια βασική απάντηση ενός πράκτορα.

## 2. Επισκόπηση αρχιτεκτονικής

### 2.1 High-level pipeline

Η διαδικασία πραγματοποιήθηκε ως ένα ενιαίο LangGraph StateGraph που ενορχηστρώνει τέσσερα λογικά στάδια:

1. **Ανάκτηση:** Με παράλληλο τρόπο οι `web_agent` και `rag_agent` εκτελούνται από τον κόμβο `START`. Κάθε ένας ανακτά πληροφορίες από τη δική του πηγή (τον δημόσιο ιστό έναντι ενός εσωτερικού διανυσματικού χώρου αποθήκευσης εργασιών `arXiv`) και γράφει την απάντησή του στην κοινόχρηστη κατάσταση.
2. **Ψηφοφορία** - τρεις πράκτορες κριτών (`strict`, `lenient`, `devils_advocate`) ταξινομούν ανεξάρτητα την απόφαση δρομολόγησης. ένας τέταρτος πράκτορας `neutral_calibrator` συγχωνεύει τις ετυμηγορίες τους σε μία μόνο ετικέτα.
3. **Σύνθεση** (πράκτορας κλήσης εργαλείων) - ο πράκτορας σύνθεσης διαβάζει την ετυμηγορία του ψηφοφόρου και τη χρησιμοποιεί για να καλέσει ακριβώς ένα από τα τρία εργαλεία (`get_only_web`, `get_only_rag`, `get_both`) που παράγουν την απάντηση που απευθύνεται στον χρήστη.
4. **Έξοδος** - η τελική απάντηση ρέει πίσω στο γράφημα και αποδίδεται στο περιβάλλον χρήστη `Streamlit` παράλληλα με ζωντανές ενημερώσεις κατάστασης.



Εικόνα2.1: Σύστημα Πολλαπλών LLM-Πρακτόρων για Αυτόματη Έρευνα και Σύνθεση Πληροφορίας

## 2.2 Αντιστοίχιση με κλασικές έννοιες πολλαπλών πρακτόρων

Κλασικές

έννοιες

Πως υλοποιήθηκαν σε αυτό το project

|                                |                                                                                                                                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Agent role / BDI</b>        | Κάθε κόμβος έχει το δικό του <code>system_prompt</code> που ορίζει τον ρόλο του (τι θεωρεί έγκυρα στοιχεία), τις επιθυμίες του (τον στόχο αξιολόγησής του) και τις προθέσεις του (ποιο εργαλείο/έξοδο θα παράγει). |
| <b>FIPA-style messaging</b>    | Το <code>LangGraph</code> λειτουργεί ως ένα κανάλι μηνυμάτων: οι κόμβοι «αποστέλλουν» μια εντολή ενημέρωσης και «λαμβάνουν» διαβάζοντας από το <code>Graph State[...]</code> .                                     |
| <b>Contract Net</b>            | Τα τρία “tools” του synthesizer που είναι ανταγωνιστικά. Το LLM ενεργεί ως διαχειριστής, αναθέτοντας σε όποιο εργαλείο πρέπει να πάει με βάση την καλύτερη ετυμηγορία του ψηφοφόρου.                               |
| <b>Disagreement resolution</b> | Υλοποιημένο ως σύνολο παράλληλης ψηφοφορίας: τρεις κριτές με σκόπιμα διαφορετικές «προσωπικότητες» + ένας βαθμονομητής που είναι σαν διαιτητής.                                                                    |

## 2.3 Σχεδιασμός κοινόχρηστης κατάστασης

Όλοι οι πράκτορες λειτουργούν με ένα μόνο σχήμα `LocalState`, το οποίο είναι ένα `TypedDict` (επεκτείνοντας το `LangGraph MessagesState` έτσι ώστε ο πράκτορας του εργαλείου synthesizer να κληρονομεί το απαιτούμενο πεδίο μηνυμάτων):

```
from typing import NotRequired
from langgraph.graph import MessagesState
```

```
class LocalState(MessagesState):
    user_question: NotRequired[str]
    rag_agent_answer: NotRequired[str]
    web_agent_answer: NotRequired[str]
    voter_agent_answer: NotRequired[str]
    synthesizer_agent_answer: NotRequired[str]
    final_answer: NotRequired[str]

    # Voting sub-graph fields
    strict_review: NotRequired[str]
    lenient_review: NotRequired[str]
    devil_review: NotRequired[str]
    final_vote: NotRequired[str]
```

**Απόφαση σχεδιασμού:** Ενιαίο κοινόχρηστο σχήμα. Οι πρώτες εκδόσεις χρησιμοποιούσαν σχήματα ανά πράκτορα (`VotingState`, `TempState`) και μεταφράζονταν μεταξύ τους σε κάθε όριο. Αυτό προκαλούσε αναντιστοιχίες κλειδιών και σφάλματα χρόνου εκτέλεσης γιατί έγινε ενοποίηση και τα πάντα μπήκαν σε ένα `LocalState`, το οποίο έκανε τη ροή δεδομένων ελεγχόμενα και αφαίρεσε μια ολόκληρη κατηγορία σφαλμάτων "ο κόμβος επιστρέφει λάθος κλειδί".

Όλα τα πεδία είναι προαιρετικά επειδή συμπληρώνονται σταδιακά καθώς εκτελείται το γράφημα. Η main function που είναι ο εντοπιστής ορίζει μόνο το user\_question. Όλα τα άλλα συμπληρώνονται από τους κόμβους.

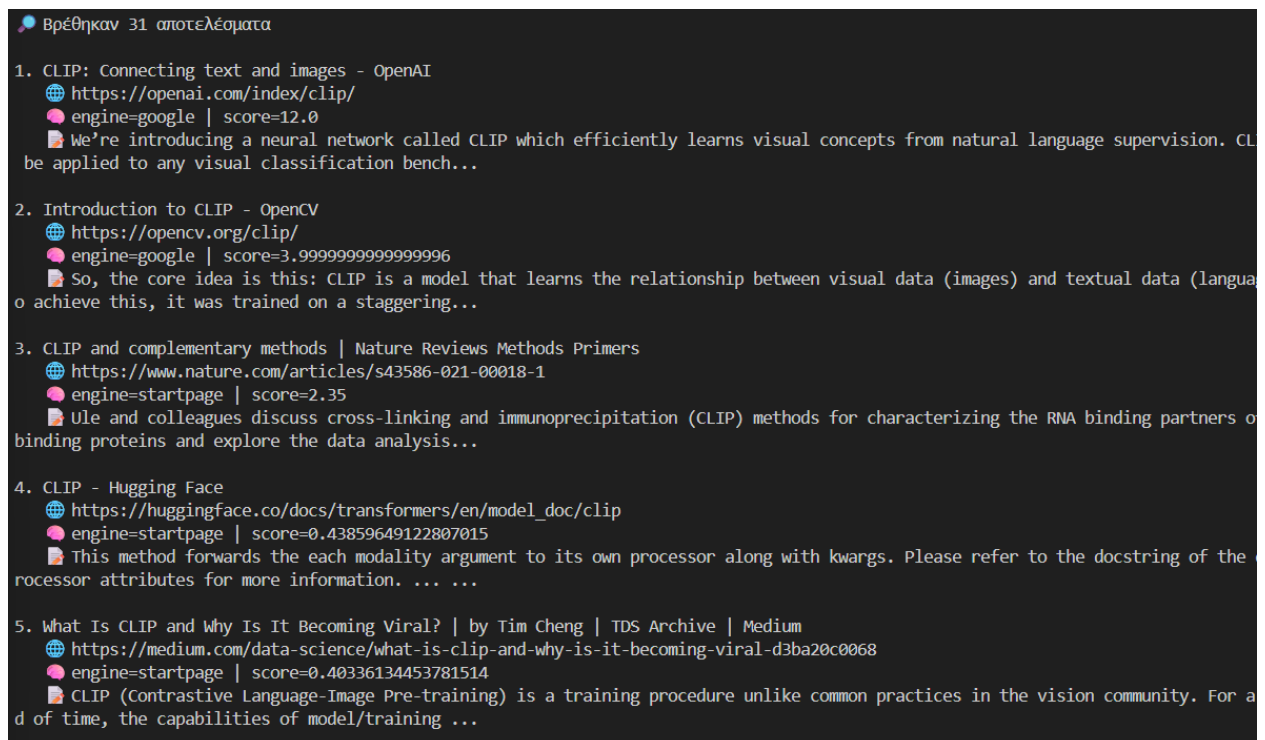
### 3. Οι Πράκτορες

#### 3.1 Web Agent

Ο web agent είναι ένας agent langchain τύπου create\_agent με δύο εργαλεία: **local\_web\_search** για αναζήτηση πολλαπλών μηχανών (Google + DuckDuckGo + Startpage), επιστρέφοντας αποτελέσματα με rank. Μετά γίνεται **fetch\_url** και γίνεται ανάκτηση του πλήρους περιεχόμενου μιας επιλεγμένης διεύθυνσης URL.

*Η έρευνα των δεδομένων γίνεται σε ένα local SearXNG το οποίο έχω κάνει deploy τοπικά ώστε να μην κάνω χρήση έτοιμων συναρτήσεων web search της OpenAI*

Για λόγους ασφαλείας (να μην μπει το μοντέλο σε ατέρμονα βρόγχο) πάρθηκαν κάποιες αποφάσεις: αναζήτηση μία φορά, επιλογή της πιο σχετικής διεύθυνσης URL, ανάκτηση μία φορά, επιστροφή σε μία μόνο εναλλακτική λύση εάν η ανάκτηση αποτύχει και ποτέ μην υπερβαίνετε τις τέσσερις συνολικές κλήσεις εργαλείου. Αυτή η οριοθετημένη εξερεύνηση ήταν μια σκόπιμη απόφαση για να διατηρηθεί η συμπεριφορά του agent προβλέψιμη και να περιοριστεί το κόστος/καθυστέρηση.



Εικόνα.3.1: Local\_web\_search αποτελέσματα επιστροφής εργαλείου

## 3.2 RAG Agent

Ο πράκτορας RAG ανακτά τα  $k$  πιο κοντινά κομμάτια από τον χώρο αποθήκευσης διανυσμάτων Qdrant. Η Qdrant περιέχει 5 άρθρα σχετικά με το Machine Learning από το arXiv. Το ερώτημα ενσωματώνεται με το sentence-transformers/all-MiniLM-L6-v2 (το ίδιο μοντέλο που χρησιμοποιείται κατά την εισαγωγή, επομένως τα vectors των embeddings ταιριάζουν), τα κομμάτια συγχωνεύονται σε ένα ενιαίο μπλοκ και το αποτέλεσμα διαβιβάζεται σε ένα LLM με μια προτροπή του συστήματος να του περιορίζει την απάντηση μόνο στο υλικό από την Qdrant και όχι από την προσωπική του βάση.

Δεν αφήνουμε σκόπιμα τον πράκτορα RAG να αποφασίσει εάν θα ανακτήσει. Ανακτά πάντα  $k=10$  κομμάτια. Επιτρέπεται στο LLM να παραλείπει την αναφορά των κομματιών εάν δεν συσχετίζονται με την ερώτηση.

```
[DEBUG] 19:57:28 ▶ local web search called | query='CLIP method'
[DEBUG] 19:57:30 ▶ Qdrant returned 10 results
[DEBUG] 19:57:30 ▶ [0] score=0.3440 | preview: 'large-scale contrastive learning paired with the image encoder
 $\mathcal{I}$ . As a result, the CLIP constraint
preserves the coherence of the fine-tuned text embedding and the
original image domain, ensuring capability and generalization....'
[DEBUG] 19:57:30 ▶ [1] score=0.3107 | preview: 'We include more ablation studies on denoising scheduler and steps and
reward weights in the supplementary material.

!["Ablation on reward models and the effect of CLIP constraint." The
"leftmost" column shows original images. Their averaged Aesthetics,
PickScore, and HPSv2 scores are 5.49, 18.19, and 0.2672, respectively.
For the following columns, we show the synthesized images "without" and
"with" CLIP constraint using different reward models. The reward scores
are listed at the bottom.
](figures/collapsev2.pdf){#fig:ablate-clip-constraint
width="\\linewidth"}...
[DEBUG] 19:57:30 ▶ [2] score=0.2913 | preview: 'To achieve this, we set CLIP space similarity as an always-online
constraint as follows,  $\begin{equation}$ 
\\small

$$\mathcal{R}_{CLIP} = \text{cosine-sim}(\mathcal{I}(\hat{\mathbf{x}}), \mathcal{I}_{\varphi}(\mathbf{p})),$$

\\label{eq:clip-constraint}
\\end{equation} and ensure  $\gamma_{\text{CLIP}} > 0$  in
eqn. eq:loss_total. Specifically, we maximize the cosine
similarity between the textual embeddings and image embeddings. The
textual embedding is obtained in forward propagation, while the image
embedding is calculated by sending the predicted image
 $\hat{\mathbf{x}}$  to the image encoder  $\mathcal{I}$  of CLIP. The
original text encoder  $\mathcal{I}_{\varphi}$  is pre-trained in
large-scale contrastive learning paired with the image encoder...
[DEBUG] 19:57:30 ▶ [3] score=0.2893 | preview: 'Recent breakthroughs in text-to-image diffusion models have brought
about a revolution in content
generation .
Among these models, the open-sourced Stable Diffusion (SD) has emerged
as the "de facto" choice for a wide range of applications, including
image editing, super-resolution, and video
synthesis .
Though trained on large-scale datasets, SD still holds two major
challenges. "First", it often produces images that do not align well
with the provided prompts .
"Second", generating visually pleasing images frequently requires
multiple runs with different random seeds and manual prompt
refinement to address
```

Εικόνα 3.2: Αποτελέσματα RAG

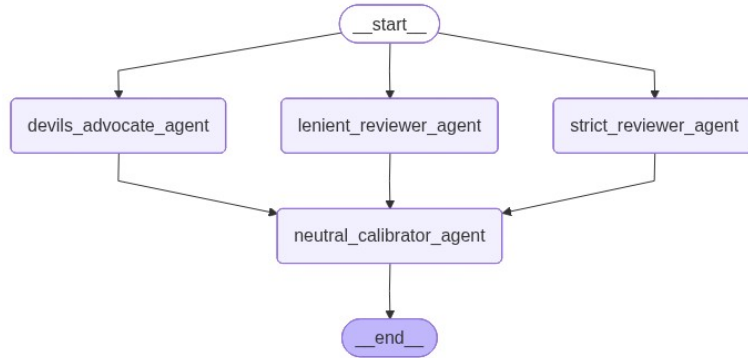
## 3.3 Voting Sub-graph

Το στάδιο ψηφοφορίας είναι το ίδιο StateGraph που αναφέρθηκε στην αρχή του εξωτερικού γραφήματος, ένα subgraph που επαναχρησιμοποιεί το ίδιο σχήμα LocalState. Υλοποιεί τον μηχανισμό επίλυσης διαφωνιών μεταξύ των πρακτόρων.

Τρεις κριτές εκτελούνται παράλληλα από την ΕΝΑΡΞΗ:

- **Αυστηρός κριτής:** εφαρμόζει υψηλό όριο πριν συστήσει την πηγή ανάκτησης δεδομένων. Έχει σαν προεπιλογή το ΚΑΜΙΑ όταν είναι αβέβαια τα αποτελέσματα που έλαβε από τους web-rag agents.
- **Επιεικής κριτής:** πρακτικός και ανεκτικός, πρόθυμος να συστήσει κάποιο σύστημα ανάκτησης.

- **Συνήγορος του διαβόλου:** αντιφατικός. αναζητά ρητά λόγους για να μην ανακτήσει.



**Εικόνα 3.3: Graph of voting agents orchestration**

Κάθε κριτής εξάγει ένα αντικείμενο JSON με ένα σταθερό σχήμα:

```

{
  "label": "ONLY_RAG" | "ONLY_WEB" | "NEITHER" | "BOTH",
  "confidence": 0-100,
  "reason": "one short paragraph",
  "key_points": ["...", "...", "..."]
}

```

Ένας τέταρτος πράκτορας είναι ο ουδέτερος βαθμονομητής ο οποίος λαμβάνει και τις τρεις αξιολογήσεις και παράγει μια ενιαία τελική ετυμηγορία στο ίδιο σχήμα. Η προτροπή του βαθμονομητή τον καθοδηγεί να αποφύγει να είναι ασυνήθιστα αυστηρός ή επιεικής και να προτιμήσει την ελάχιστη επαρκή μέθοδο.

**Σχεδιασμός:** τρεις προσωπικότητες + διαιτητής αντί για ψήφο πλειοψηφίας. Μια ψήφος καθαρής πλειοψηφίας θα απέρριπτε τη συλλογιστική κάθε κριτή και θα χρησιμοποιούσε μόνο τις ετικέτες. Ζητώντας από τον βαθμονομητή να διαβάσει το πλήρες JSON κάθε αξιολόγησης (συμπεριλαμβανομένης της αιτιολογίας και των key\_points), μπορεί να διευθετήσει περιπτώσεις όπου, π.χ., ο αυστηρός κριτής επέλεξε ΚΑΜΙΑ από τις δύο με αδύναμο λόγο, αλλά ο επιεικής και υπέρμαχος του διαβόλου επέλεξε ΚΑΙ ΤΑ ΔΥΟ με ισχυρούς λόγους. Ο βαθμονομητής ζυγίζει τα επιχειρήματα, όχι μόνο τις ψήφους.

Έξοδος μόνο ένα JSON. Το SYSTEM\_PROMPT για κάθε κριτή επιβάλλει αυστηρή έξοδο JSON (χωρίς περιορισμούς, χωρίς προοίμιο, ακριβή ονόματα πεδίων). Αυτό καθιστά την ανάλυση ντετερμινιστική και μας επιτρέπει να αποφύγουμε την εύθραυστη εξαγωγή που βασίζεται σε regex.

```

[{"timestep": 1, "role": "lenient_reviewer_agent", "output": {"label": "ONLY_WEB", "confidence": 80, "reason": "The query about CLIP requires both internal knowledge about its functionality and external information regarding its origin and applications, making a combination of internal and external sources necessary.", "key_points": ["CLIP combines text and image features.", "It enables zero-shot learning tasks.", "Information is sourced from both internal and external articles."]}, "timestamp": "2023-03-04"}, {"timestep": 2, "role": "devils_advocate_agent", "output": {"label": "ONLY_WEB", "confidence": 80, "reason": "The query about CLIP requires both internal knowledge and recent web information, as it involves technical details and context that may be better covered in external sources.", "key_points": ["CLIP combines text and image features.", "It enables zero-shot learning tasks.", "Information is available in both internal and web articles."]}, "timestamp": "2023-03-04"}, {"timestep": 3, "role": "strict_reviewer_agent", "output": {"label": "NEITHER", "confidence": 70, "reason": "The query about CLIP requires both internal knowledge about its function and external verification of its origin, as the user specifically asks for the article it was found in.", "key_points": ["CLIP is a text-image encoder used in models like Stable Diffusion.", "It allows for zero-shot learning by combining image and text features.", "The user requested the source article, necessitating web search."]}, "timestamp": "2023-03-04"}, {"timestep": 4, "role": "neutral_calibrator_agent", "output": {"label": "ONLY_WEB", "confidence": 70, "reason": "The query about CLIP requires both internal knowledge about its function and external verification of its origin, as the user specifically asks for the article it was found in.", "key_points": ["CLIP is a text-image encoder used in models like Stable Diffusion.", "It allows for zero-shot learning by combining image and text features.", "The user requested the source article, necessitating web search."]}, "timestamp": "2023-03-04"}]

```

**Εικόνα 3.4: Αποτελέσματα βαθμολογιών**



### 3.4 Synthesizer Agent

Ο synthesizer είναι ένας πράκτορας κλήσης εργαλείων που έχει κατασκευαστεί με το `create_agent` (όπως και όλοι οι άλλοι agents). Έχει τρία εργαλεία:

- **get\_only\_web**: επιστρέφει την απάντηση του web agent αυτούσια.
- **get\_only\_rag**: επιστρέφει την απάντηση του RAG agent αυτούσια.
- **get\_both**: καλεί μια ξεχωριστή κλήση LLM που συγχωνεύει και τις δύο απαντήσεις σε μία μόνο απάντηση.

Και τα τρία εργαλεία χρησιμοποιούν το `InjectedState` για ανάγνωση από το `LocalState` χωρίς να εκθέτουν αυτά τα πεδία στο LLM ως ορίσματα εργαλείου. Το πρόβλημα με τα εργαλεία της `langchain` ήταν ότι ανάγκαζαν το μοντέλο να συμπληρώνει τα ορίσματα που έχουν μέσα τα εργαλεία και για αυτόν τον λόγο με το `InjectedState` το μοντέλο δεν έχει επαφή με τα αυτά.

```
@tool(return_direct=True)
```

```
def get_both(state: Annotated[LocalState, InjectedState]) -> dict:
```

```
    """Use this tool when voter says BOTH web search AND RAG are important. """
```

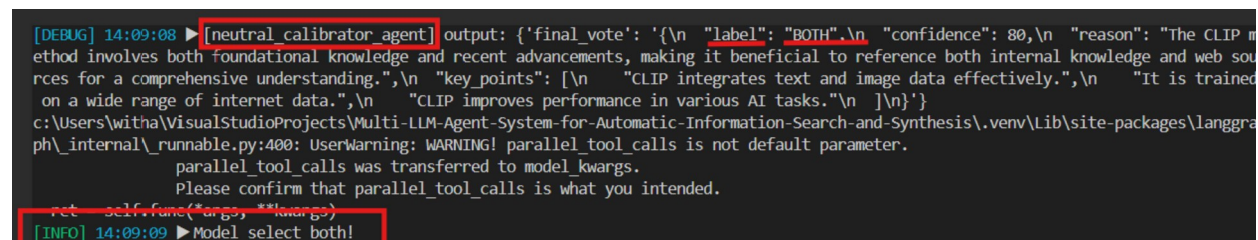
```
    user_prompt = create_user_prompt(state)
```

```
    messages = af.build_messages(user_prompt=user_prompt, system_prompt=SYSTEM_PROMPT,
langchain_model_flag=True)
```

```
    result = ms.openai_model().invoke(messages["messages"])
```

```
    return {"final_answer": result.content}
```

Η ετυμολογία του ψηφοφόρου μεταβιβάζεται στον συνθέτη ως το μήνυμα του χρήστη σαν είσοδο του πράκτορα. Οπότε ο συνθέτης λαμβάνει την ερώτηση του χρήστη συνδυασμένη με την απάντηση του ψηφοφόρου (το τελικό αποτέλεσμα το οποίο εμπεριέχει το `label` και το `reason`) και με βάση αυτό μπορεί να επιλέξει ανάμεσα στις τρεις περιπτώσεις (tools) που αναλύθηκαν προηγουμένως



```
[DEBUG] 14:09:08 ► [neutral_calibrator_agent] output: {'final_vote': {'label': "BOTH",\n "confidence": 80,\n "reason": "The CLIP method involves both foundational knowledge and recent advancements, making it beneficial to reference both internal knowledge and web sources for a comprehensive understanding.",\n "key_points": [\n "CLIP integrates text and image data effectively.",\n "It is trained on a wide range of internet data.",\n "CLIP improves performance in various AI tasks." ]}}\n c:\\Users\\witha\\VisualStudioProjects\\Multi-LLM-Agent-System-for-Automatic-Information-Search-and-Synthesis\\.venv\\Lib\\site-packages\\langgraph\\_internal\\runnable.py:400: UserWarning: WARNING! parallel_tool_calls is not default parameter.\n parallel_tool_calls was transferred to model_kwargs.\n Please confirm that parallel_tool_calls is what you intended.\n [INFO] 14:09:09 ► Model select both!
```

Εικόνα 3.5: Απόφαση `Neural_calibrator_agent` και αναγνώριση του synthesizer ότι πρέπει να λάβει υπόψη και τις δύο

## 4. Βασικές Αποφάσεις Υλοποίησης

### 4.1 Απενεργοποίηση παράλληλων κλήσεων εργαλείων

Τα μοντέλα OpenAI έχουν από προεπιλογή την `parallel_tool_calls=True`, η οποία επέτρεπε στο synthesizer LLM να καλεί δύο εργαλεία μίας πηγής παράλληλα (π.χ., `get_only_web` και

get\_only\_rag μαζί) αντί να επιλέγει ένα. Αυτή η τεχνική χρησιμοποιήθηκε σε όλους τους agents εκτός του synthesizer ο οποίος πρέπει να επιλέξει ΜΟΝΟ ένα εργαλείο και μετά να επιστρέψει την απάντηση.

**Διόρθωση:** ορισμός parallel\_tool\_calls=False στο μοντέλο που χρησιμοποιείται από τον synthesizer:

```
def non_parallel_openai_model():  
    return ChatOpenAI(  
        model=settings.api_model,  
        api_key=settings.open_api_key,  
        temperature=0.0,  
        streaming=True,  
        parallel_tool_calls=False,  
    )
```

Αυτή ήταν μια δομική διόρθωση, ακόμα κι αν η προτροπή είναι ασαφής, το μοντέλο δεν μπορεί πλέον να καλέσει πολλά εργαλεία ταυτόχρονα.

#### 4.2 return\_direct=True είναι απαραίτητο αλλά όχι επαρκές

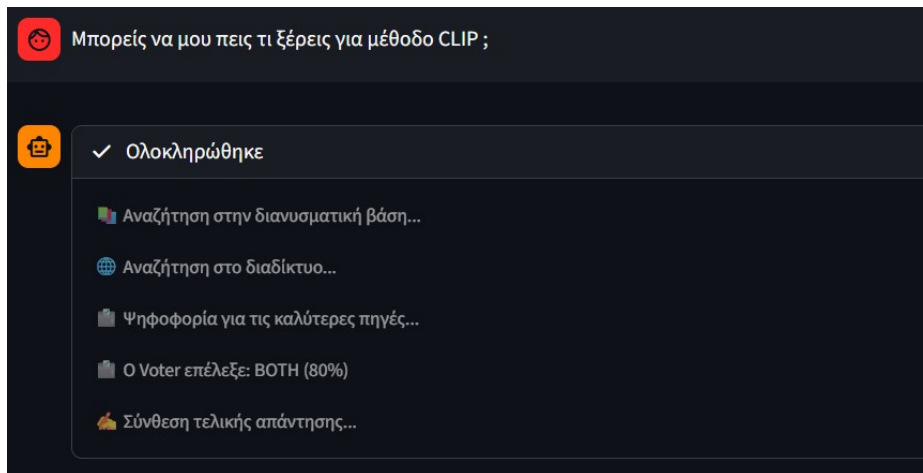
Βάλαμε και στα τρία εργαλεία synthesizer την μεταβλητή return\_direct=True, έτσι ώστε ο πράκτορας να τερματίζεται μετά την εκτέλεση του εργαλείου (χωρίς να μπορεί να κάνει μεταφορά σε άλλο εργαλείο). Ωστόσο, το return\_direct αποτρέπει μόνο μια επόμενη επανάληψη του βρόχου πράκτορας, δεν εμποδίζει την εκτέλεση πολλαπλών εργαλείων εντός του ίδιου κόμβου εργαλείων όταν το μοντέλο εκπέμπει παράλληλες κλήσεις. Γι' αυτό ήταν επίσης απαραίτητη η §4.1.

#### 4.3 Ροή του γραφήματος για σχόλια στο περιβάλλον χρήστη

Ο μηχανισμός ενορχήστρωσης ξαναγράφηκε ώστε να αποδίδει συμβάντα κατάστασης καθώς ολοκληρώνονται οι κόμβοι για να υπάρχει έλεγχος και παρακολούθηση της ροής:

```
for chunk in compiled.stream({"user_question": user_question}, stream_mode="updates"):  
    for node_name, node_output in chunk.items():  
        yield {"type": "status", "node": node_name, "label": NODE_LABELS[node_name]}  
        if node_output is not None:  
            final_state.update(node_output)  
    yield {"type": "final", "answer": final_state["synthesizer_agent_answer"]}
```

Το Streamlit μέσω του st.status(...), παρέχει μία ετικέτα κάθε βήματος ως γκρι κείμενο «λεζάντα» σε πραγματικό χρόνο. Οι χρήστες βλέπουν ποιος agent εργάζεται αυτήν τη στιγμή αντί για ένα κενό loading spinner.



Εικόνα 4.1: Σχόλια ροής γραφήματος

#### 4.4 Εξαγωγή τελικής απάντησης

Η τελική απάντηση του synthesizer μπορεί να βρεθεί σε δύο σημεία: στο πεδίο κατάστασης `final_answer` ή στο περιεχόμενο του τελευταίου `ToolMessage`. Με στόχο να μην υπάρχει ευθραυστότητα στις διαφορές έκδοσης του LangGraph, εξάγουμε το αποτέλεσμα με έναν ποιο ευέλικτο τρόπο:

```
model_answer = final_state.get("final_answer")
if not model_answer:
    for msg in reversed(final_state["messages"]):
        if isinstance(msg, ToolMessage) and msg.content:
            try:
                model_answer = json.loads(msg.content)["final_answer"]
            except (json.JSONDecodeError, KeyError):
                model_answer = msg.content
            break
```

Αυτό το μοτίβο απέδειξε την αξιοπιστία του σε δύο εκτελέσεις κατά τις οποίες συμπληρώθηκε το πεδίο κατάστασης και σε μία εκτέλεση συμπληρώθηκε μόνο το πεδίο `ToolMessage`.

## 5. Evaluation: Multi-agent vs. Single-agent Baseline

Για να αξιολογήσουμε εάν η αρχιτεκτονική πολλαπλών πρακτόρων προσφέρει καλύτερα αποτελέσματα, έγινε σύγκριση του πολυπρακτορικού συστήματος με το μονοπρακτορικό σύστημα (δηλαδή μια απλή κλήση στο chatbot). Το μοντέλο σε όλες τις περιπτώσεις ήταν το GPT-4o-mini με την ίδια ερώτηση του χρήστη χωρίς ανάκτηση, χωρίς ψηφοφορία, χωρίς σύνθεση.

Τις δοκιμές τις κάναμε με την ακόλουθη ερώτηση, η οποία επιλέχθηκε επειδή έχει τόσο ανακτήσιμο εσωτερικό πλαίσιο (μια εργασία arXiv που εμπεριέχει η διανυσματική βάση) όσο και το τρέχον εξωτερικό πλαίσιο (Wikipedia):

«Τι γνωρίζετε για τη μέθοδο CLIP; Μπορείτε επίσης να μου πείτε την εργασία από την οποία πήρατε τις πληροφορίες;»

| Criterion          | Single-agent baseline                                                                     | Multi-agent system                                                              |
|--------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Source attribution | Αναφέρει μόνο μια πηγή                                                                    | Παραθέτει τόσο την εργασία arXiv όσο και τη Wikipedia, με URL                   |
| Coverage           | Γενική περιγραφή                                                                          | Συνδυάζει τεχνικές λεπτομέρειες (RAG) με το τρέχον πλαίσιο (ιστός)              |
| Hallucination risk | Μεγαλύτερο hallucination καθώς προσπαθεί να απαντήσει με ότι ξέρει από την εκπαίδευση του | Κάθε πληροφορία εντοπίζεται σε ένα πραγματικά ανακτημένο κομμάτι ή ιστοσελίδα   |
| Latency            | ≈2-3 s                                                                                    | ≈25-30 δευτ. (παράλληλη ανάκτηση + 3 κριτές + βαθμονομητής + synthesizer)       |
| Token cost         | 1 call                                                                                    | 9-10 calls                                                                      |
| Failure visibility | Το μοντέλο παράγει μόνο κείμενο                                                           | Η συλλογιστική κάθε πράκτορα καταγράφεται και εμφανίζεται στο περιβάλλον χρήστη |

**Συμπέρασμα:** Το σύστημα πολλαπλών πρακτόρων ανταλλάσσει την καθυστέρηση και το κόστος με την αξιοπιστία της απάντησης. Για μια ερώτηση ερευνητικού τύπου όπου ο χρήστης θέλει να μάθει σωστά αποτελέσματα από έγκυρες πηγές αντί να λάβει μια γρήγορη αλλά λάθος απάντηση.

### Multi-Agent RAG

Τι γνωρίζετε για τη μέθοδο CLIP; Μπορείτε επίσης να μου πείτε την εργασία από την οποία πήρατε τις πληροφορίες;

✓ Ολοκληρώθηκε

Η μέθοδος CLIP (Contrastive Language-Image Pretraining) είναι μια τεχνική που αναπτύχθηκε για να εκπαιδεύει νευρωνικά δίκτυα χρησιμοποιώντας ζεύγη εικόνας και κειμένου. Αυτή η μέθοδος επιτρέπει στο μοντέλο να προβλέπει το πιο σχετικό κείμενο για μια δεδομένη εικόνα, χωρίς να απαιτεί άμεση βελτιστοποίηση για συγκεκριμένες εργασίες. Η CLIP χρησιμοποιεί μια απλή εργασία προεκπαίδευσης, όπου το μοντέλο μαθαίνει να προβλέπει ποια λέξη/τάξη αντιστοιχεί σε ποια εικόνα, αξιοποιώντας μια μεγάλη βάση δεδομένων με 400 εκατομμύρια ζεύγη εικόνας-κειμένου που συλλέχθηκαν από το διαδίκτυο.

Η μέθοδος CLIP διατηρεί τη συνοχή μεταξύ των κειμενικών και των εικονικών ενσωματώσεων, εξασφαλίζοντας ικανότητα και γενίκευση. Στην εργασία "TextCraftor: Your Text Encoder Can be Image Quality Controller" αναφέρεται ότι η ομοιότητα στο χώρο CLIP χρησιμοποιείται ως μια συνεχής περιοριστική συνθήκη, με στόχο τη μεγιστοποίηση της συσχέτισης μεταξύ των κειμενικών και εικονικών ενσωματώσεων.

Μετά την προεκπαίδευση, το μοντέλο μπορεί να χρησιμοποιηθεί για διάφορες εργασίες υπολογιστικής όρασης, όπως αναγνώριση αντικειμένων και γειω-τοποθέτηση, χωρίς να απαιτείται εκπαίδευση σε συγκεκριμένα σύνολα δεδομένων. Η μέθοδος έχει αποδειχθεί ότι είναι ανταγωνιστική με παραδοσιακά μοντέλα που απαιτούν επιβλεπόμενη εκπαίδευση.

Η πληροφορία αυτή προέρχεται από τις εργασίες "Learning Transferable Visual Models From Natural Language Supervision" και "TextCraftor: Your Text Encoder Can be Image Quality Controller".

Τι γνωρίζετε για τη μέθοδο CLIP; Μπορείτε επίσης να μου πείτε την εργασία από την οποία πήρατε τις πληροφορίες;

Η μέθοδος CLIP (Contrastive Language-Image Pretraining) είναι ένα μοντέλο που αναπτύχθηκε από την OpenAI και παρουσιάστηκε σε μια εργασία το 2021 με τίτλο "Learning Transferable Visual Models From Natural Language Supervision". Η CLIP συνδυάζει εικόνες και κείμενα, επιτρέποντας στο μοντέλο να κατανοεί και να συσχετίζει οπτικές και γλωσσικές πληροφορίες.

Η βασική ιδέα πίσω από τη CLIP είναι η εκπαίδευση ενός μοντέλου που μπορεί να αναγνωρίζει εικόνες με βάση περιγραφές κειμένου. Αυτό επιτυγχάνεται μέσω μιας διαδικασίας αντίθετης εκπαίδευσης (contrastive learning), όπου το μοντέλο μαθαίνει να συσχετίζει εικόνες με τις αντίστοιχες περιγραφές τους, ενώ ταυτόχρονα απορρίπτει τις μη σχετικές περιγραφές.

Η CLIP έχει δείξει εξαιρετική απόδοση σε διάφορες εργασίες αναγνώρισης εικόνας και μπορεί να γενικεύσει σε νέα δεδομένα χωρίς να απαιτεί επιπλέον εκπαίδευση. Αυτό την καθιστά ιδιαίτερα χρήσιμη για εφαρμογές που απαιτούν κατανόηση και αναγνώριση περιεχομένου σε εικόνες με βάση φυσική γλώσσα.

Αν θέλετε περισσότερες λεπτομέρειες ή συγκεκριμένες πιτυχίες της μεθόδου, μη διστάσετε να ρωτήσετε!

Εικόνα 5.1: Αριστερά είναι το πολυπρακτορικό σύστημα ενώ δεξιά το μονοπρακτορικό

## 6. Η Στρατηγική Ψηφοφορίας: Παράμετροι και Αιτιολόγηση

Οι παράμετροι που επιλέχθηκαν για την στρατηγική ψηφοφορίας:

- Τρεις κριτές, όχι δύο ή πέντε. Δύο κριτές παράγουν ισοπαλίες πολύ συχνά. Οι πέντε αυξάνουν το κόστος χωρίς αναλογικό όφελος. Τρεις με διαφορετική προσωπικότητα

(αυστηρός, επιεικής, συνήγορος του διαβόλου) δίνουν στον βαθμονομητή τρεις ανεξάρτητες προοπτικές.

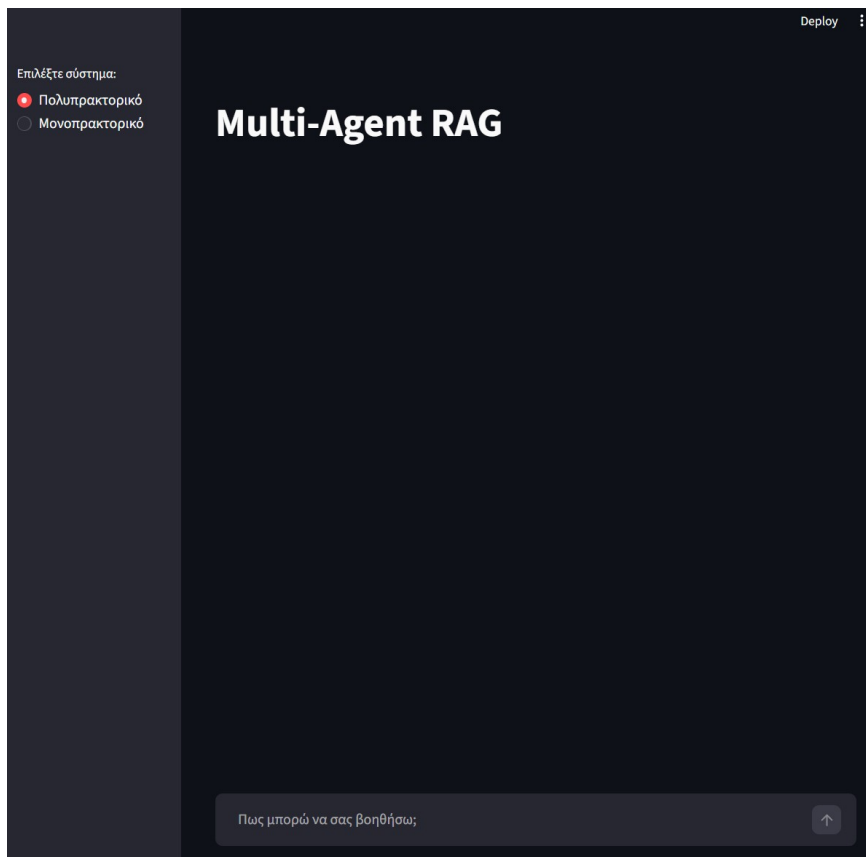
- Βαθμονομητής αντί για ψήφο πλειοψηφίας. Όπως συζητήθηκε στην §3.3, ο βαθμονομητής διαβάζει ολόκληρο το σκεπτικό, όχι μόνο τις ετικέτες, καθιστώντας το ανθεκτικό σε ασθενώς δικαιολογημένες πλειοψηφίες.
- Η εμπιστοσύνη καταγράφεται αλλά δεν χρησιμοποιείται για δρομολόγηση. Εξετάσαμε τη στάθμιση των κριτών με βάση την εμπιστοσύνη, αλλά το "75 έναντι 85" ενός μικρού μοντέλου δεν βαθμονομείται ουσιαστικά. Η εμπιστοσύνη καταγράφεται για εντοπισμό σφαλμάτων.
- Τέσσερις ετικέτες (ONLY\_RAG, ONLY\_WEB, BOTH, NEITHER). Η επιλογή NEITHER δηλώνει ότι δεν είναι απαραίτητη η επιλογή κάποιας και επιτρέπει στο σύστημα να απαντά σε ασήμαντες ερωτήσεις ("αναδιατύπωση αυτής της παραγράφου") χωρίς περιττή ανάκτηση. Στις δοκιμές που έγιναν ενεργοποιείται σωστά σε ερωτήσεις που δεν χρειάζονται εξωτερικές πληροφορίες.

Σε όλες τις ερωτήσεις του τεστ, ο βαθμονομητής συμφώνησε με τον επιεική κριτή τις περισσότερες φορές ( $\approx 80\%$  των περιπτώσεων), με τον αυστηρό και τον συνήγορο του διαβόλου λιγότερο. Ο βαθμονομητής δεν θα πρέπει να επιλέγει απλώς την προκατάληψη ενός μόνο κριτή, και πράγματι δεν το κάνει.

## 7. Streamlit User Interface

Η τελική διεπαφή χρήστη αποτελείται από:

1. Το `st.chat_input` για την ερώτηση του χρήστη.
2. Μόνιμο ιστορικό μηνυμάτων μέσω `st.session_state.messages`.
3. Ένα `st.status` που μεταδίδει ενημερώσεις κατάστασης με γκρι κείμενο καθώς κάθε agent που ολοκληρώνεται ( Αναζήτηση στην διανυσματική βάση...,  Αναζήτηση στον διαδίκτυο...,  Ψηφοφορία για τις καλύτερες πηγές...,  Σύνθεση τελικής απάντησης...).
4. Η τελική απάντηση αποδίδεται με `st.markdown` μόλις ολοκληρωθεί ο synthesizer.
5. Κουμπί εναλλαγής `st.radio` από πολυπρακτορικό σύστημα σε μονοπρακτορικό για τις δοκιμές



Εικόνα7.1: Συνολική εικόνα συστήματος

## 8. Συμπεράσματα

Κατασκευάστηκε ένα λειτουργικό σύστημα πολλαπλών πρακτόρων που ανακτά δεδομένα από δύο διαφορετικές πηγές παράλληλα, επιλύει διαφωνίες μέσω ενός πρωτοκόλλου ψηφοφορίας τριών κριτών + βαθμονομητή και συνθέτει μια τελική απάντηση μέσω ενός πράκτορα κλήσης εργαλείων που υπακούει στην ετυμηγορία του ψηφοφόρου. Η αρχιτεκτονική αντιστοιχίζεται με σαφήνεια στις κλασικές έννοιες πολλαπλών πρακτόρων του μαθήματος, ενώ υλοποιείται σε σύγχρονα πλαίσια (LangGraph, LangChain, OpenAI).

Τα περισσότερα προβλήματα/σφάλματα παράχθηκαν στην προσπάθεια αντιστοίχισης κλειδιών μεταξύ των πρακτόρων. Η ενοποίηση όλων κάτω από ένα LocalState και η χρήση του MessagesState ως το parent class εξάλειψε ολόκληρη αυτήν την κατηγορία σφαλμάτων. Η βασική γραμμή ενός πράκτορα παράγει απαντήσεις που είναι εν μέρη καλές. Το σύστημα πολλαπλών πρακτόρων παράγει απαντήσεις που μπορούμε να εμπιστευτούμε, επειδή κάθε ισχυρισμός είναι ανιχνεύσιμος.

## Appendix A - Δομή του Project

```
└── streamlit_app.py          # User interface
```

```
|— all_agents_orchestration.py    # Outer LangGraph orchestrator (generator)
|— local_state.py                 # Shared LocalState schema
|— web_agent.py                   # Web search agent
|— rag_agent.py                   # RAG agent over Qdrant
|— voting_agent_orchestration.py  # Voting sub-graph (4 reviewers)
|— synthesizer.py                 # Tool-calling synthesizer agent
|— model_storage.py               # ChatOpenAI factory functions
|— prompt_storage.py             # All system/user prompts in one place
|— agent_functions.py            # build_messages and helpers
|— utils/
|   |— settings.py               # Pydantic settings (API keys, URLs)
|   |— logs.py                  # Colored logger
```